

whereas transformer encoder is an untrained brain!

← (a pretrained brain that already understands language)

BERT → feature extractor (context aware)
(add task specific heads)

Date _____
Page _____

Day 23 (07/04/26)

BERT (Bidirectional Encoder Representation from Transformer)

• Pretrain • Finetuning

How to train encoder part of transformer to make it BERT?

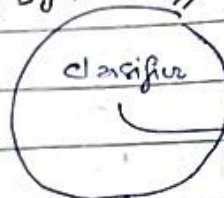
→ word prediction

no sentence generation
kills if B follows A //

(i) Mask Language Modeling

(ii) Next Sentence Prediction

(Sentence relationship classification)

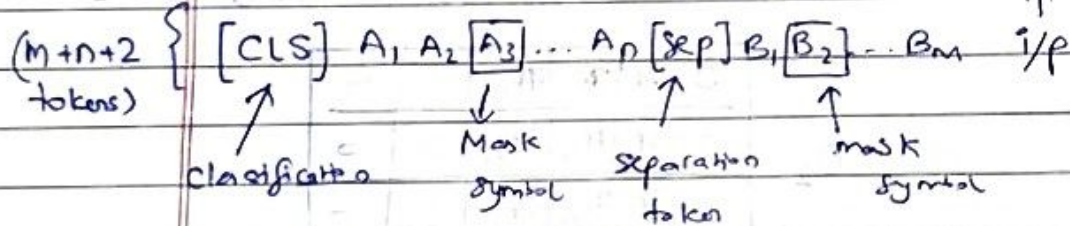
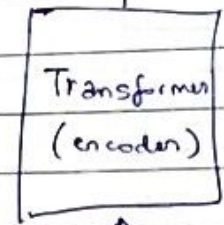


→ 2 sentences fed first

(A, B)

as 2 tasks done in parallel

N words M words



NOTE: MLM head
NSP head

- (i) position encoding (embedding)
- (ii) sentence identifier — segment embedding
- (iii) embedding (word)

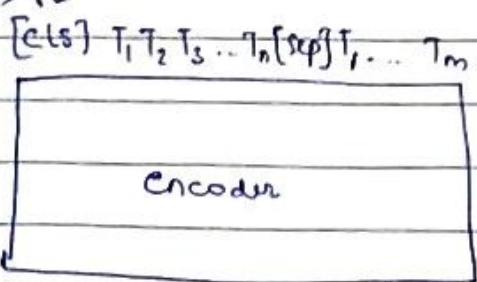
Summation of these 3 = input
↓ gives
K, Q, V

only for learning language!!
during training time
during finetuning new head!!

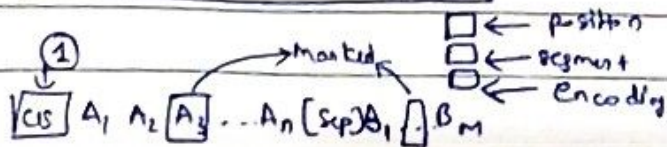
⇒ how to get (i) and (ii) P ⇒ learnable vector (while training time)

(we need to choose here)

this vector is used for classifier if B follows A
(0 1 ⇒ yes
else ⇒ no)



Each of these are vectors of corresponding input vector (contextual embedding)



(check how done for both tasks)

classmate

Date _____
Page _____

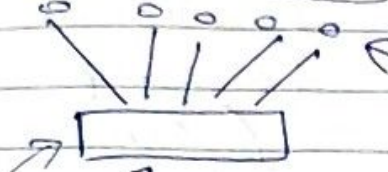
Training Part explained

to do Mark language prediction (the word)

softmax

helps in vocabulary prediction

(Pretraining)



word corpus

(for classification - only 2 words)

I think par

word (output side)

given to classifier for NSP

0 not next sentence
1 yes

NOTE: [CLS] tells if B follows A

(used for Next Sentence Prediction)

CLS embedding given → binary level

→ No. of tokens limited by the vectors (i), (ii), and (iii)

Finetuning

why CLS? it captures whole sentence meaning

→ acts like summary vector!!

→ only 1 sentence given this time and then with CLS

embeddings → vector, it is predicted → after passing to linear layer/softmax,

→ for Q and A → we will have Q in input side

for output → we get A

[CLS] Q [SEP] Passage [SEP]

Some half

model predicts

So basically → what we have to do is: whatever token

Start and end position

is generated pass it to classifier for

of the answer inside

predicting the exact word!!

the passage!!

Pass the i/p to encoder → It gives token

↓

to know what word it is, pass

it to classifier!!

to know if classification/NSP → pass CLS to determine 1 or 0

→ multi model (varia)
→ image representation
Image Embedding → (VGG-16), AlexNet, ResNet
(Models) ↑ model

→ Convolution
then
pooling

(Seq of it)

Introduced

Skip

Connection

or residual
Connection

→ it may lead to
deeper network

→ Stable gradients!

→ faster convergence!

→ something related to
backprop (check)

→ Inception (In all 3 we generally see the depth)
↓

- here horizontal also seen (horizontal expansion)
- 1x1 convolution present
- 3x3, 5x5 also

~~skip?~~ # Network Embedding → (How to get the embeddings for
graphs!)

→ Real world pb can be represented in form of network

→ non-linear representation (mostly linear representation as
is sequential) but if we
want to (NLR) then NE

→ network → (NE)

(Vector) (representation)

→ Node Embedding

→ Edge "

maintains network
Structure

in embedding space !!

(PCA representation)

Day 24 (08/04/26) - (Graph ML)

Graph Types

(i) regular graph

- all nodes have same no. of connections (degree)
- ↳ high clustering coefficient (CC)
- ↳ usually longer paths

⇒ Very uniform networks

↓
not realistic

for real world networks

(ii) random graph

- connections are randomly formed
- ↳ low clustering coefficient
- ↳ short average path length

⇒ no structure but surprisingly efficient (short paths)

(iii) Small-world networks

→ combines

↳ high clustering (like regular graph)

↳ Short paths (like random graph)

example: Social networks (6 degrees of separation)

(any 2 people connected through 6 intermediate ppl)

- few long-range connections (break ties) reduce distance

(iv) scale free networks

- Some nodes have very high degree (hubs)

↳ follows power law distribution

↳ few nodes = highly connected

example: internet, social media

- these hubs dominate ~~most~~ connectivity

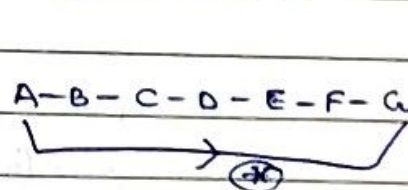
↙ basis of link prediction: which missing edges are likely to form!

(v) triadic closure: If A connected B, A connected to C

⇒ Then B and C likely connected

Types: a) weak tie: connection but yet formed

b) Strong tie: triangle completed



(vi) clustering coefficient (CC)

↳ measure how connected are my neighbours

→ local CC (node level)

↳ for a node,

$$CC(n) = \frac{\text{actual triangles}}{\text{possible triangles}}$$

→ global CC (graph level)

↳ average over all nodes

→ High CC → tightly knit community

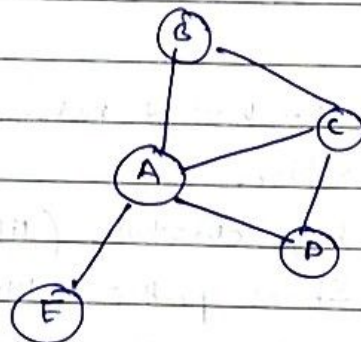
• Link Prediction

→ problem: given a graph → predict future or missing links/edges

→ based on: (i) triadic closure

(ii) Common neighbors

(iii) structural similarity



$$CC(A) = \frac{2}{4C_2}$$

→ example: friend suggestion (facebook, linkedin)

→ (hand crafted importance

• Centrality features used before embedding are)

↳ how important is a node in the network

serves as
main feature

⇓ (No embeddings)!!

Types: ① Degree centrality

↳ no. of connections a node has

$$C_D(n) = \text{degree of node } n$$

↳ more connections → more important

↳ ignores global structure

↳ fast, simple

NOTE: old: graph $\rightarrow$ centrality $\rightarrow$ ML
 modern: graph $\rightarrow$ embedding $\xrightarrow[\text{node}]{\text{classmate}} \rightarrow$ ML

$\rightarrow$ good for viral marketing

(i) Closeness centrality

$\rightarrow$ how close a node is to all others

$$C_c(u) = \frac{1}{\sum \text{Short distances}}$$

$\rightarrow$ Smaller distance $\Rightarrow$ faster reach $\Rightarrow$ more important

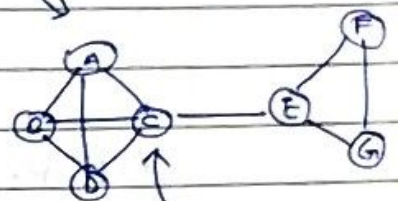
good for
dumpling
the
network

(ii) Betweenness Centrality

$\rightarrow$ how often a node lies on shortest paths

helps in restricting info to some nodes

$$C_b(u) = \sum \frac{\text{Paths through } u}{\text{total shortest paths}}$$



don't let
E group to
get info
from A group

$\rightarrow$ nodes acts like a bridge / network bottleneck

$\rightarrow$ very important for:

(a) attack detection

(b) network vulnerability

NOTE: (i), (ii) and (iii) doesn't account
for importance / credibility
of neighbours

(iv) Eigenvector centrality

$\rightarrow$ Importance depends on important neighbours

$\hookrightarrow$ being connected to important people makes you important

(v) Page Rank $\leftarrow$ google algorithm

$\rightarrow$ modified eigenvector centrality

$\hookrightarrow$ Idea:

(a) random walks on graph

(b) important nodes get more visits

depends on:

(i) Incoming links

(ii) importance of those
sources

(iii) their outgoing degree

$\uparrow$

As importance gets split

(vi) Katz Centrality

$\hookrightarrow$ like eigenvector but:

(a) Consider all ~~near~~ paths (not just direct neighbours)

(b) longer paths $\rightarrow$ less weight

- Eigenvector Centrality

(i) Adjacency matrix

→ Converting a graph to matrix form

$$A[i][j] = \begin{cases} 1 & \text{if edge exists} \\ 0 & \text{otherwise} \end{cases}$$

→ first step towards embeddings



Graph → Matrix → linear algebra → embeddings

(ii) Matrix Powers

ex. A^2

It tells no. of 2 hop paths between nodes

ex. $A \rightarrow B \rightarrow C$

$$A^2[A][C] = 1$$

That is A and C are 2 steps away

→ This connects to link prediction, node similarity

(iii) eigenvector centrality

↳ node importance depends on neighbour's importance

Recursive

defn

$$i.e. \quad A^T u = \lambda u$$

⇒ Importance vector influenced by adjacency matrix

NOTE: Importance (node)

= Sum of (importance of neighbours)

8 How to compute this Importance matrix P

↓ principal
This is nothing but the eigenvector
of the adjacency matrix
↓
this vector gives the importance score
for each node

Step 1: Start with random vector

Step 2: multiply by Adjacency matrix

Step 3: normalize (to avoid explosion)

Step 4: repeat

⇓
Converges → gives eigenvector centrality

Example A — B — C

part of pb
matrix

Method 1:

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

node: this will change to M

for page rank $\sum \text{imp}$
 $\Rightarrow M = \frac{\text{no. of links}}{\text{no. of this}}$

let's take $x = [1 \ 1 \ 1]$

$\Rightarrow x^0 = [1 \ 1 \ 1]^T$

$x^1 = A x^0 = [1 \ 2 \ 1]^T$

$x^2 = A x^1 = [2 \ 2 \ 2]^T$

$x^3 = A x^2 = [2 \ 4 \ 2]^T$

No λ : This is not
eigenvector

do $Ax \neq \lambda x$

but $[1 \ 2 \ 1]$ is !!

$Ax = \lambda x$

Method 2:

① $[1 \ 1 \ 1] \rightarrow$ ② $[1 \ 2 \ 1] \rightarrow$ ③ $[2 \ 2 \ 2] \rightarrow$ ④ $[2 \ 4 \ 2]$

↑ ↑ ↑ B_p ↑ ↑ ↑ B_p

$B_p \Rightarrow B_{prev}$

$B_p \Rightarrow B_{prev}$

Start anywhere
repeat: either follow link
or randomly jump!

• Page Rank

↳ importance flows through links like a random surfer

- (i) Start on random node
- (ii) Randomly follows links

Nodes visited more often $\Rightarrow$ higher Page rank

→ update rule: not just sum of importance of
neighbour nodes

(New score
= from neighbours /
their degree

but,

importance from neighbour

$\div$ its outgoing links

+ random jump)

In prev

score vector
computational!

This is done so: if a node has many outgoing
links: it spreads its importance thin

→ Page Rank depends on:

- (i) Incoming links
- (ii) Importance of source nodes
- (iii) their outgoing degree
- (iv) random jump factor $\Rightarrow$ helps in:

With some pb, do it

(a) Dangling nodes

nodes with no outgoing

links

random jump avoids
random walk to stop

(b) spider trap

↳ groups of nodes linking to each other

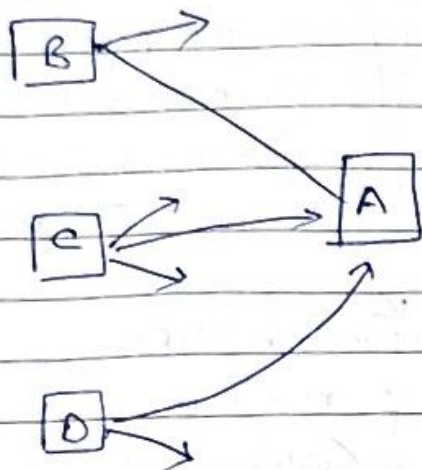
R.J avoids importance getting trapped!!

Day 25 (13/04/26)

(Simple unbiased random walk)

PageRank $\rightarrow$ reflects which page/node is highly visited (imp $\uparrow$)

$$PR(A) = \frac{PR(B)}{2} + \frac{PR(C)}{3} + \frac{PR(D)}{2}$$



$$PR(P) = \sum_{Q \rightarrow P} \frac{PR(Q)}{\# \text{out}(Q)}$$

Transition Matrix

$$\begin{matrix}
 & \begin{matrix} 1 & 2 & 3 & \dots & n \end{matrix} \\
 \begin{matrix} 1 \\ 2 \\ 3 \\ \vdots \\ n \end{matrix} & \rightarrow & \begin{bmatrix} 0 & 1/2 & 0 & \dots & 1/2 \\ 1/3 & 0 & 1/3 & 0 & 1/3 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix}
 \end{matrix}
 \cdot
 \begin{matrix} \Pi \\ \vdots \\ \vdots \end{matrix}
 =
 \begin{matrix} \Pi \\ \vdots \\ \vdots \end{matrix}$$

$A \quad \quad \quad \Pi \quad \quad \quad \Pi \Pi$

instead
of adjacency
matrix, we

$$A^T \Pi = \Pi$$

have transition matrix

prev value of page rank

(Here take A^T so to
consider incoming links)

- column vector tells incoming links
- row vector tells outgoing links

(# rich get richer)

$$\left\{ \begin{array}{l} A^T u^0 = u^1 \\ A^T u^1 = u^2 \\ \vdots \\ A^T u^{K-1} = u^K \end{array} \right\} \Rightarrow A^T \text{ should have some properties then } u^k \text{ is always the principle eigen vector}$$

(google page ranks = 50-60 iterations)

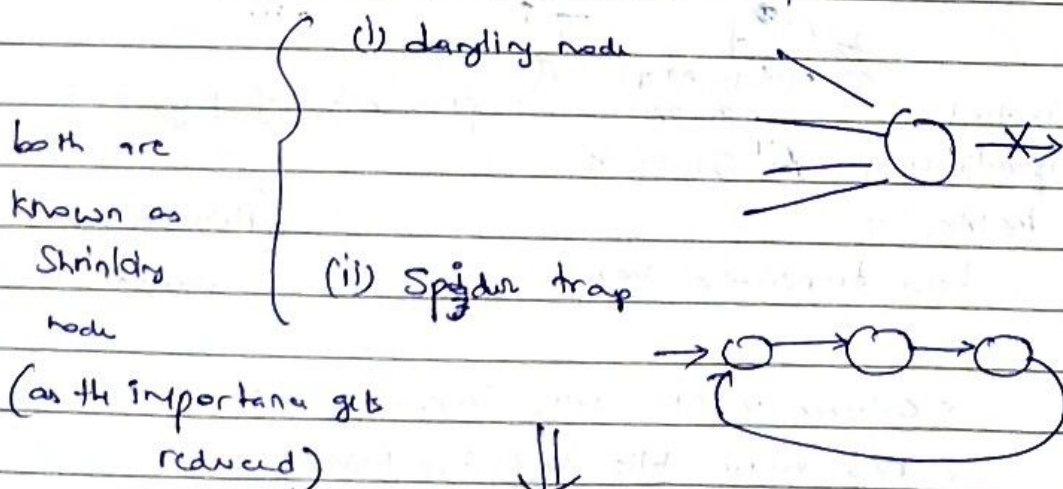
- (i) positive semi definite
 (ii) sum of row vector = 1
 (column vector also)
 it's a stochastic matrix
 $\Rightarrow$ max value of principle eigen = 1

NOTE: In real scenarios,

transition matrix may not be proportional

(equal probability) $\Rightarrow$ not converging *

$\Rightarrow$ as there are 2 scenarios:



To solve this,

• add a damping factor (0.6 - 0.8)

$$PR(p) = d \sum_{q \in \text{in}(p)} \frac{PR(q)}{\text{out}(q)} + (1-d)$$

damping factor

- for any dangling node in transition matrix add $\frac{1}{n}$
 ↳ outgoing links

$(1-d) \rightarrow$ represented by random jump $\rightarrow$ (with pb = $\frac{1}{n}$ for each node)

i.e. $G_2 = d A^T + (1-d) \begin{bmatrix} \frac{1}{n} & \frac{1}{n} & \frac{1}{n} & \dots & \frac{1}{n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{1}{n} & \frac{1}{n} & \frac{1}{n} & \dots & \frac{1}{n} \end{bmatrix}$

(with high pb) $\nearrow$ A^T $\nwarrow$ (with low pb)

$$\pi^i = d A^T + I c^T$$

$$c = \begin{bmatrix} \frac{1}{n} \\ \frac{1}{n} \\ \vdots \\ \frac{1}{n} \end{bmatrix}$$

$$\pi^i = [d A^T + (1-d) I c^T] \pi^{i-1}$$

central idea $\left\{ \begin{array}{l} \bullet \text{ follow the ~~highest~~ hyperlink with } d \text{ probability} \\ \bullet \text{ and with } (1-d) \text{ pb go to any random node} \end{array} \right.$

for personalised page rank { i.e. for an individual, which pages should be

((more visited?))
 c tells (or how each pg is weighted)

$$p = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0.2 \\ 0.1 \\ \vdots \\ 0.1 \end{bmatrix}$$

$\rightarrow$ how is it found?
 $\Rightarrow$ historical data

personalised $\rightarrow$ $\pi^i = [d A^T + (1-d) I p^T] \pi^{i-1}$
 page rank

Rooted page ranks (Instead of visiting multiple pages $\Rightarrow$ visit only 1)

$$\vec{r} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Why imp?

$\Rightarrow$ relation betⁿ a page and other page

$\Rightarrow$ done by computing page rank
// for that particular page

(i.e. if we find page rank by replacing e with r)

the importance we get is explicitly for that single page with which we want to find relation)

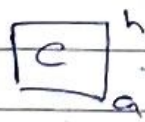
Page rank used in $\rightarrow$ (i) ^{case} search optimization
(ii) defend web spamming
(iii) web (-----)

Another paper similar to page ranks but not very popular

(~~HTS~~) (HITS)

//
Hypertext induced topic Search)

every page has 2 scores $\rightarrow$ hub
 $\rightarrow$ authority



$\rightarrow$ sort of outgoing links

hub (how resourceful a particular page is) $\Rightarrow$ # hyperlinks to important page

authority (reference: sort of how many incoming links)

$$a(p) = \sum_{q \rightarrow p} h(q) \Rightarrow A^T h = a$$

adjacency matrix
 (here transpose as column $q \Rightarrow$ incoming)

$$h(p) = \sum_{p \rightarrow q} a(q) \Rightarrow A a = h$$

no transpose as outgoing links req which is h now

$$a = A^T h$$

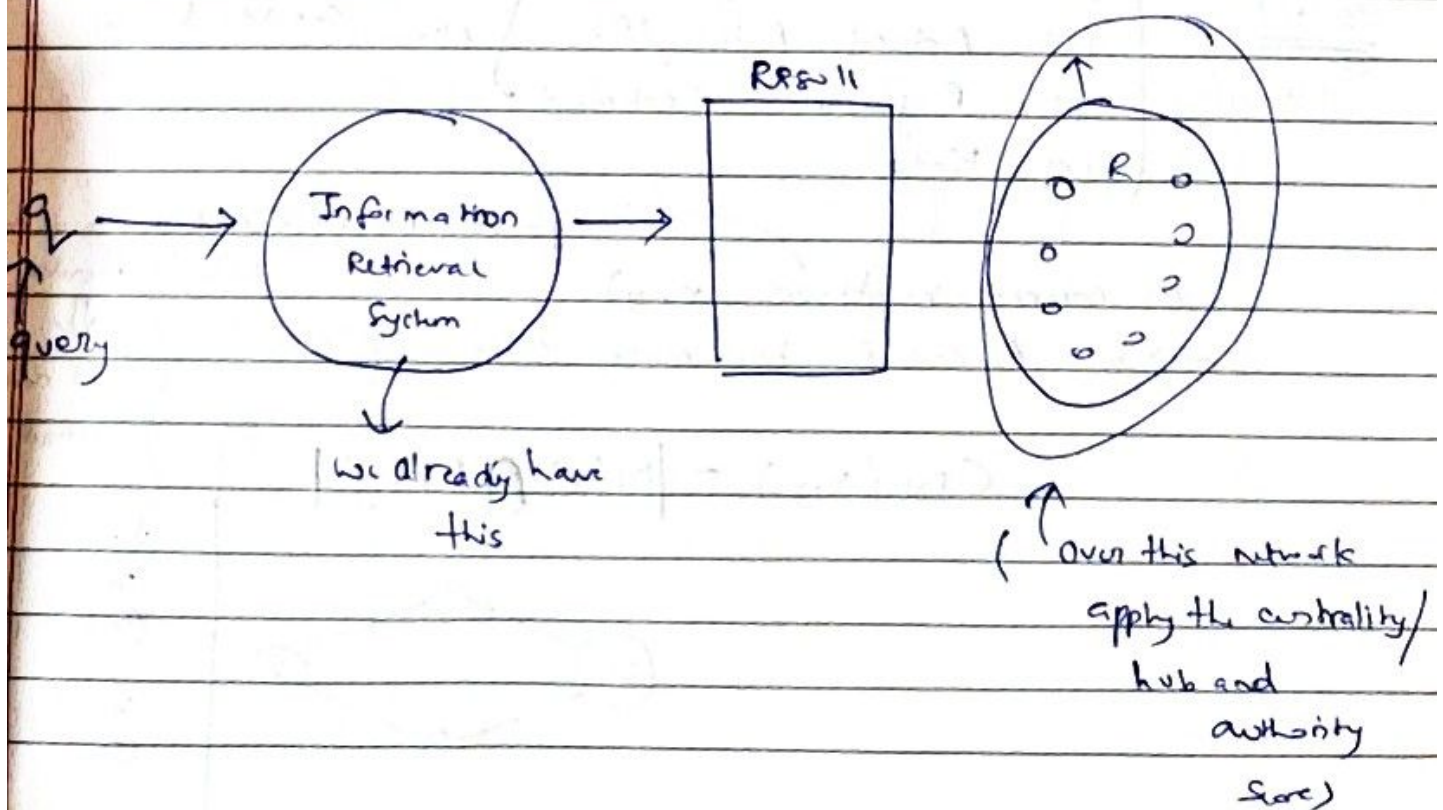
$$= \boxed{A^T A} a$$

B $\rightarrow$ eigen vector of $A^T A$ gives a

$$h = \boxed{A A^T} h$$

B $\rightarrow$ eigen vector of $A A^T$ gives h

for a search case $\rightarrow$ use hub score
 for reference case $\rightarrow$ use authority score



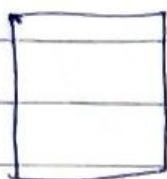
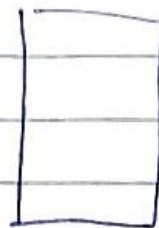
Link Prediction

{ future links or missing links }

(i) Future link prediction

(ii) Missing links prediction

ex.

Snapshot
of network
at time t  $t+1$ • What will be the new
connections at $t+1$ time?

Example. Protein-Protein Interaction network



What are the missing links?

• Common methods to solve (i) and (ii)

Uses topological
structure
features of
networks

(a) Common neighbours

(b) Jaccard Coefficient

(c) Adamic Adar / Resource Allocation

(d) Preferential attachment

(e) Katz

local networks

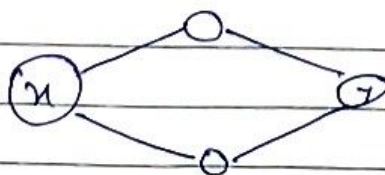
Considered only

(a) common neighbours (x, y)

→ both A and B how many common friends

pb with this: if x and y have more
casual friends

$$CN(x, y) = |N(x) \cap N(y)|$$

CN not
good metric

x	y	(X)
(1000)	(1000)	

x	y	(✓)
(5)	(4)	

(b) Jaccard coefficient

↳ solves the actual friends pb

$$JC(x, y) = \frac{|N(x) \cap N(y)|}{|N(x) \cup N(y)|}$$

← if large,

$JC \downarrow$

⇒ not good friends

Superior
than (a)
and (b)

~~Adamic Adar~~

↳ credibility of CN

↳ considers through whom (x) and (y) are connected

↳ doesn't consider the count of neighbours

$$AA(x, y) = \sum_{z \in N(x) \cap N(y)} \frac{1}{N(z) \log(N(z))}$$

no. of neighbours of z

Common neighbours of x, y

$$\text{Resource Allocation} = \sum_i \frac{1}{N(z)}$$

(d) Preferential Attachment (used for network modelling)

⇕

$$pb(x) \cdot pb(y)$$

$$|N(x)| \cdot |N(y)|$$

also for this prediction

pb of x making friends
and pb of y making friends

Day 26 (16/04/2026) → (sort of classification pb)
(Link Prediction old)

from
topological
perspective
(this concept)

← (e) Katz

(f) Rooted Page Rank — for finding relation betw nodes

(g) Hitting Time / Commute Time

(h) SimRank (x, y)

$$= \frac{C}{|N(x)||N(y)|} \sum_{\substack{a \in N(x) \\ b \in N(y)}} \text{simrank}(a, b) \quad \text{similar to page ranks}$$

• hitting time (x, y) or no. of hops
= ~~shortest~~ time to pass
info. from x to y

→ (not symmetric betw x to y
and y to x)

• Commute time

$$= HT(x, y) + HT(y, x)$$

- recursive method
- termination condition?

$$\hookrightarrow \text{simrank}(x, x) = 1$$

$$0 < C < 1$$

↑ damping/decay
factor

(e) Katz Link Prediction

(no. of path length l)

o for link prediction start from l=2

o to check how strong,
and for generalization

Start from

l=1

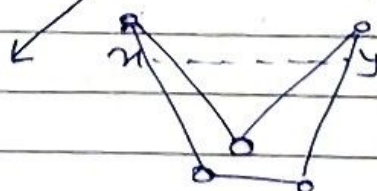
Katz betw all

$$\text{pairs} : \beta^1 A + \beta^2 A^2 + \beta^3 A^3 + \beta^4 A^4 + \dots$$

$$\beta \in [0, 1] = 0.5$$

let's say :-

- 2 nodes
- want to predict if link exists/
can occur



• Compute all
shortest
paths
(2 hops only) first

• then calculate all diff lengths

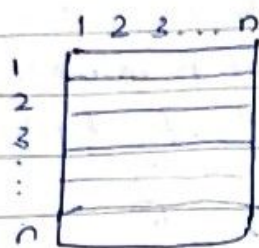
Idea:

→ give less
weights to
large paths

Connection with
2 hops

How to solve it as classification pb

(i) first do vectorisation



A



(PL) ← It can be very sparse

← no. of nodes may be large

↑ curse of 10^7 (10^9)
dimensionality

how to reduce this?

(a) PCA? → gives $n \times k$

→ not scalable in terms of large network

(b) Can we do feature engineering?

→ Can we represent a node by some features?

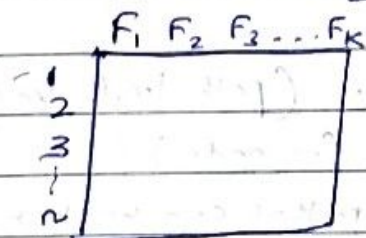
given a node $\Rightarrow$ (i) Cluster Coefficient

(ii) degree

(iii) close

(iv) between

(v) any topological
features



pb? $\Rightarrow$ clear defⁿ of ~~feature~~ feature

feature suitability ambiguity

~~Wrt~~ Wrt pb

(# Network Embedding)

(i) embedding (networks) LSI, CUR

(ii) matrix factorisation — PCA, SVD, EVD etc

(iii) ~~word~~ embedding methods on graph — Deepwalk, Node2vec

(iii) message passing — graph neural networks

(variation of GNN) { — GCN

— GAT

→ graph attention network

Condition of a embedding method requires?

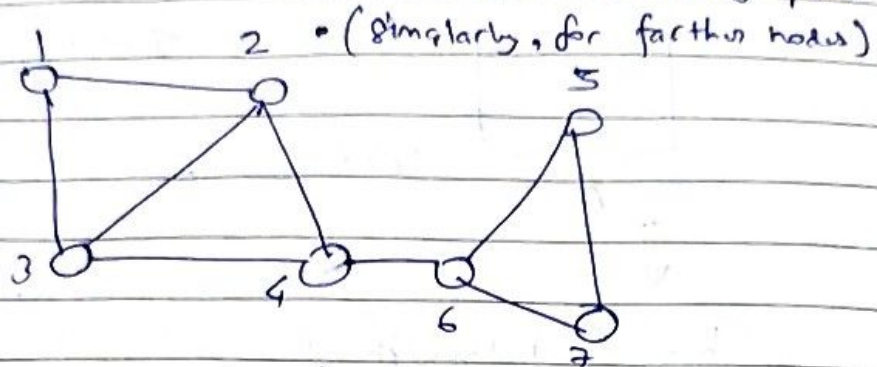
good
embedding

→ Close nodes in network after

embedding should be closer to each other

→ in embedding space

(PCA captures this!)



→ (both bfs, dfs used to generate seqs.)

(b) Word Embedding method

→ random walks from a node

→

```

1 2 3 4 6
1 3 2 4
3 1 2
  
```

1	2
3	4
	6 5
	7

• generate diff sequences (path length varying)

• they act as ^{seq of} words (or nodes)

• any word embedding method can work here

Can we apply skip gram?

(Yes)

→ (word2vec), transform etc

Can work!!

⇒ unsupervised approach!! → CBOW

→ since we don't know underlying method

and no parameters learnt

(Pls)

Seq will be centralised

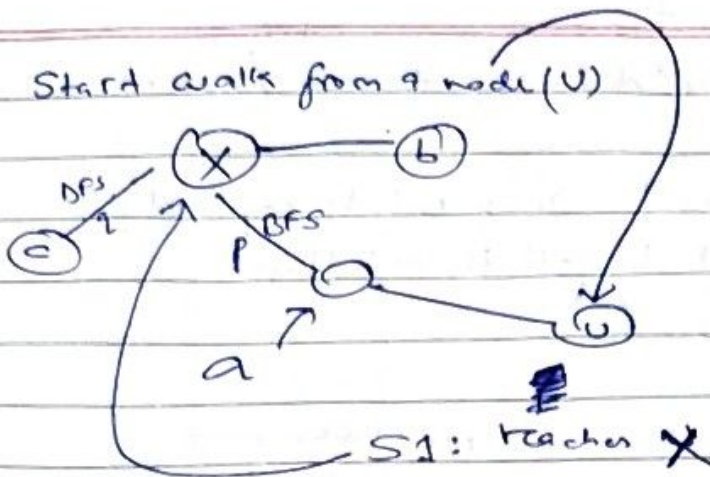
on imp nodes

→ Page ranks, eigenvector centrality

← if unbiased random walks ⇒ method is known as deep walk, then apply skip gram

→ biased random walks ⇒ word2vec
that's why!!

So: Start walk from a node (U)

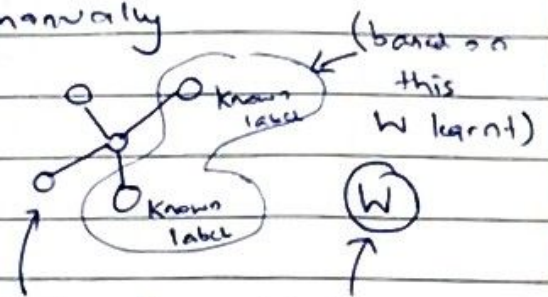


• DFS and BFS are done with diff

ph, p and q

• this is determined by manually

• not used anymore

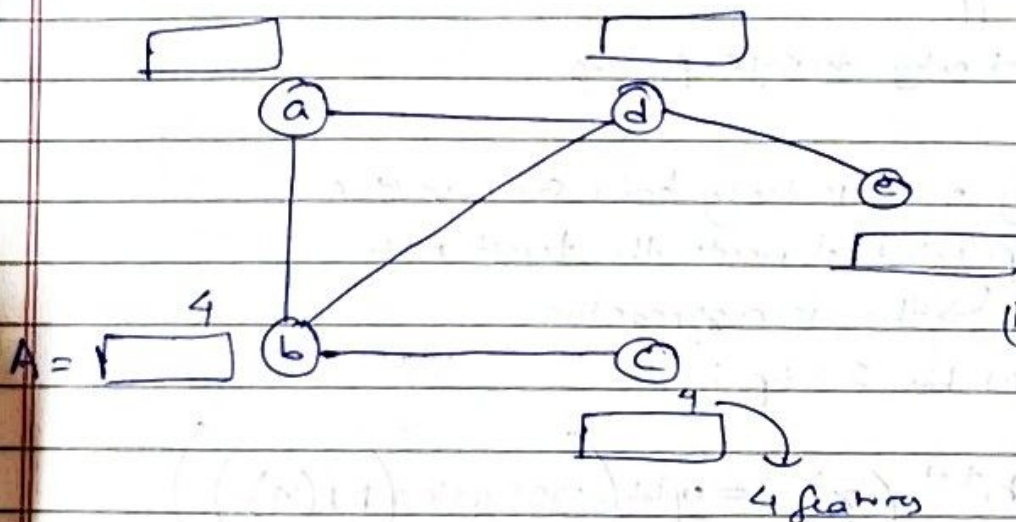


(c) Graph Neural Network

→ supervised method

→ based on certain underlying tasks, the model learns

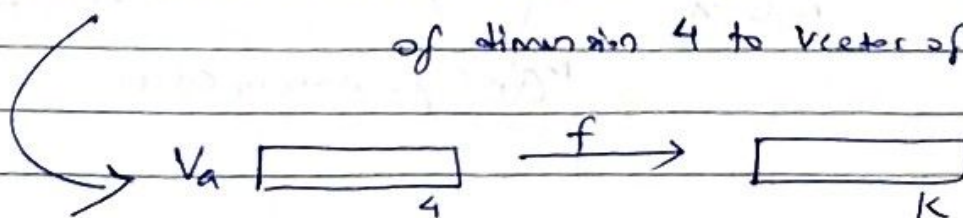
Q What is the task?



(i) message passing
to capture the
Structure of
Network

(ii) Assumption:
we have a vector
representation
of the network
Initially!!

Want to do: Convert the A vector
of dimension 4 to vector of K

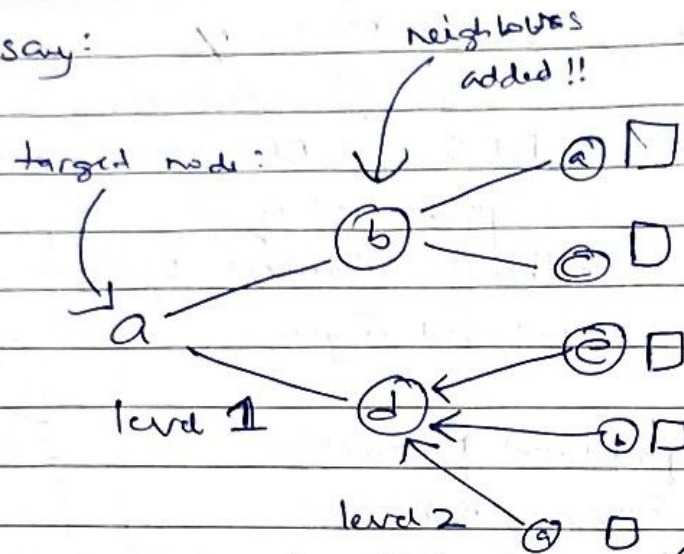


the f^n

• Why not apply f directly to each ^{node} vector?

- ↳
- Graph structure may not be captured!!
 - even though we get the embeddings

let's say:



- keep on extending till k levels (preferred is 2 or 3)

IDEA: • Structural property taken from neighbours!!



↳ by message passing

→ every node initially have a vector

→ Propagated towards the target node

↳ this is aggregation

GNN has 2 steps:

$$h^{t+1}(x) = \text{update} \left(\text{Aggregate} \left(\underline{\underline{N(x)}} \right) \right)$$

$\uparrow$ node x $\uparrow$ $\uparrow$ $\uparrow$ neighbours of x

major functions in GNN

$$h^{t+1}(x) = \text{Update} \left[h^t(x) + \underset{y \in N(x)}{\text{Aggregate}} (h^t(y)) \right] ? \text{ (not sure) (check) if true}$$

• What are aggregate fn?

- Average of all nodes
- Max
- Sum

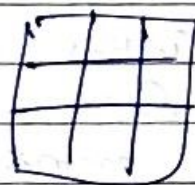
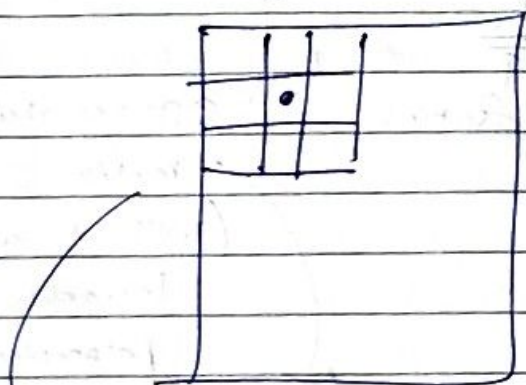
variation of GINN

- depends on variations of aggregation and update fn

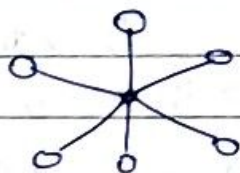
• What all update fn?

- $h^t(x) + \text{aggregate}$

extension of GINN to Convolution



In convolution,
→ 3x3 is replaced by one value



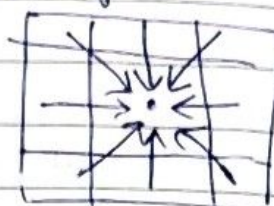
⇒ Visualizing image as graph
→ a node is a pixel with adjacent pixel as neighbour node

GNN

(network added) (convolution)



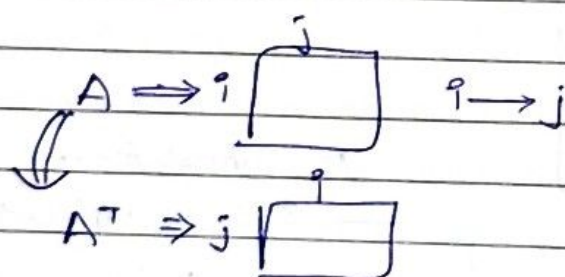
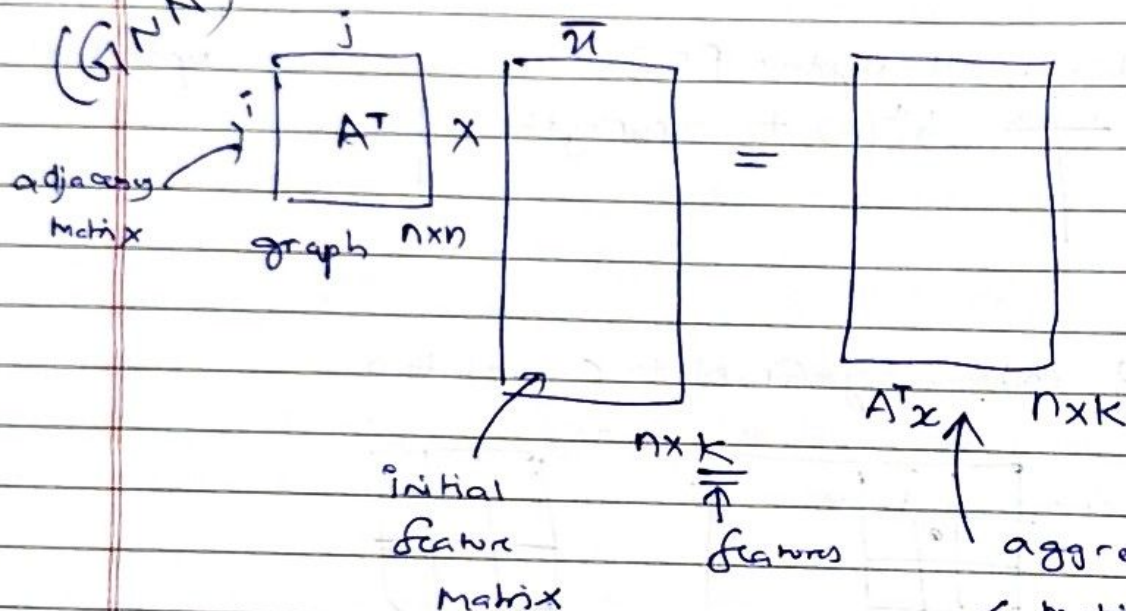
sort of 'like'



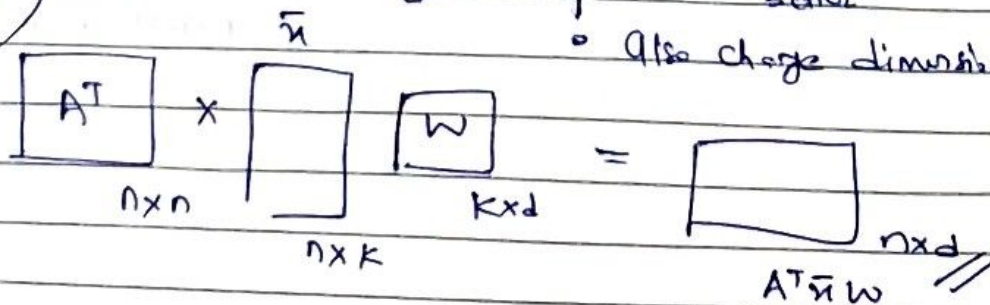
instead of

Simple $v_c \Rightarrow$ can we have $W v_c$? $\Rightarrow$ so that W can be learnt? $\circ$ Another ~~network~~ network added

(GNN)

aggregated matrix
(without any learnable parameters) $\circ$ this is sort of embedding of the network $\circ$ this we want to get better $\circ$ also change dimension

(GNN)



GNN:

$$h^{t+1}(u) = \sigma \left(W h(u), \sum_{y \in N(u)} W h^t(y) \right)$$



NOTE: Sort of $h^t(u)$, $h^t(y)$ etc given to a

neural network $\rightarrow$ so transformed!!
and then added (or updated!!)

o GNN of 1 layer

$\Rightarrow$ propagation from just immediate neighbours

we don't go further than 2 or 3 layers

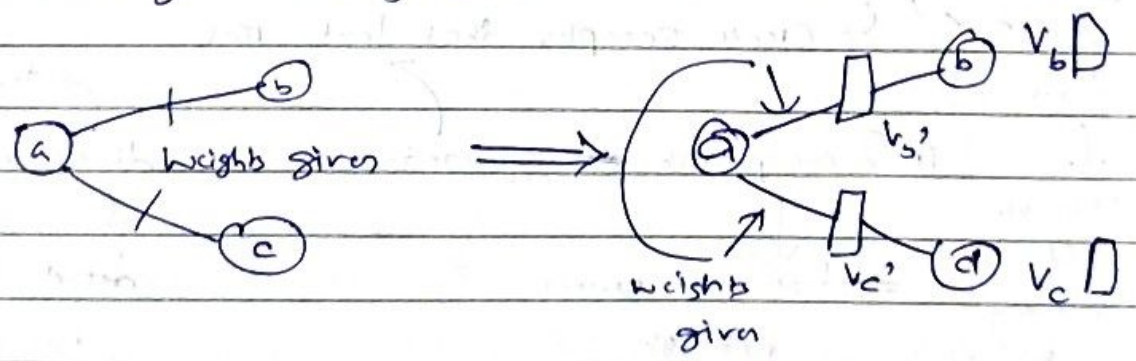
What about GAT?



Similar to attention within words

$\rightarrow$ do attention within neighbours

$\rightarrow$ adds weight to neighbours



to give weights $\Rightarrow$ first define score (similarity score etc)

$$e_{da}^t = a(W h_d^t, W h_a^t)$$

$\Rightarrow$ defines score for connected vectors (similarity)

let's say: a is parametrised product (attention)

S: first do convolution $\rightarrow$ define w_h^t

So: Define score: e_{ij} (for the updated feature vector)

S1: then apply softmax over e_{ij}

S2: do the aggregation!!

Graph Transformer

↓
generate sequences
↓
pairs to transformer

Basic idea:

(GNN)

- o aggregate from neighbors
- o update

NOTE: we discussed only homogeneous network!!

How to take care of overfitting - normalization

Applications - will be discussed

Day 27 (21/04/26)

• Generative Adversarial Networks

→ used for multimedia (not good in text domain)

(Just a framework) → Create samples that looks real

↓
where we can fit any networks

(i) 2 components - generator

↓
⇒ both are neural networks

→ traditional classifier
discriminator

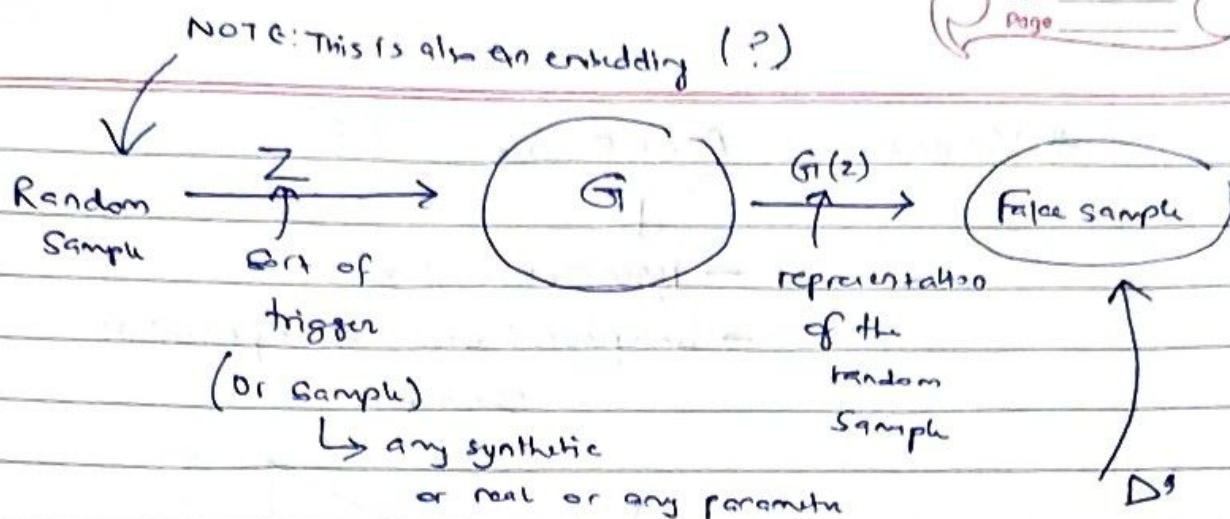
↓
detects if sample real or not

(ii) 2 types of sample - real data

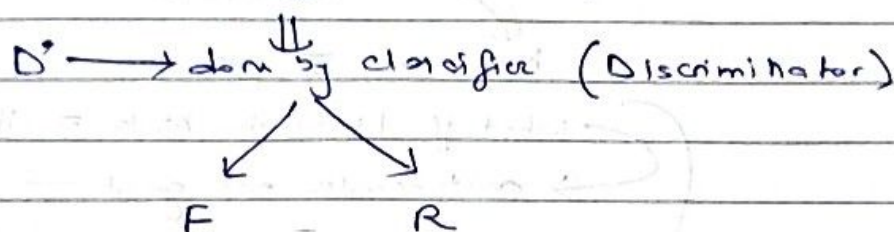
random sample
can have multi label images

few of these req by (D)

random sample ← used by (G) to create samples



Idea: Make D and D' close //



1) first train discriminator

(ii) give random sample $\rightarrow$ generate an image $\rightarrow$ feed to Discriminator

(iii) If detected false, generator needs to be updated

$\Downarrow$

$\rightarrow (1 - \epsilon)$ (misclassification)

loss function given by discriminator

If detected real, output of discriminator used for

ϵ updating discriminator

} repeated

Which network to use for generator and discriminator?

how to generate fake samples?

$\rightarrow \text{Eval}_{data(u)}[\log D(u)]$

$\rightarrow$ Discrimination: maximising $\textcircled{u}$ is real //

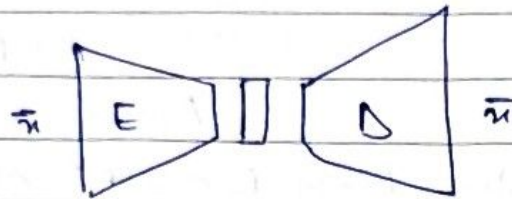
Which loss functions used? - (mentioned in slide)

$\downarrow$

Note in case of generator $\rightarrow$ maximise first term
minimise second term } (!)

Variational AutoEncoder (?)

- regeneration of x
- unsupervised way of representation generation

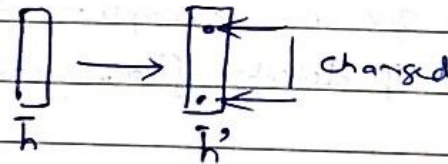


Autoencoder
here only z generated

What if I want to make z like output?
→ autoencoder not good

- as z points to a single embedding

for z like things, change the embedding



If done, $z \approx z' \rightarrow$ (not guaranteed)

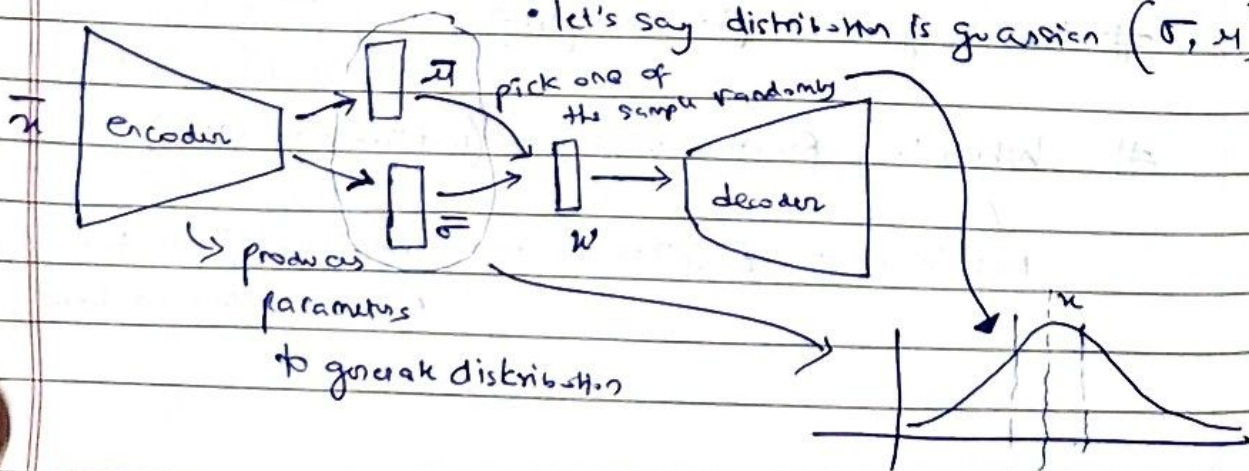
To handle this
Variational autoencoder

⇒ generate a distribution P_z

Instead of
a single
embedding

how to generate it?

- let's say distribution is gaussian (μ, σ)



#

$$x \longrightarrow z \longrightarrow p(z|x) \sim q(z|x) \quad (?)$$

$$\Downarrow$$

$$\theta \leftarrow (\mu, \sigma)$$

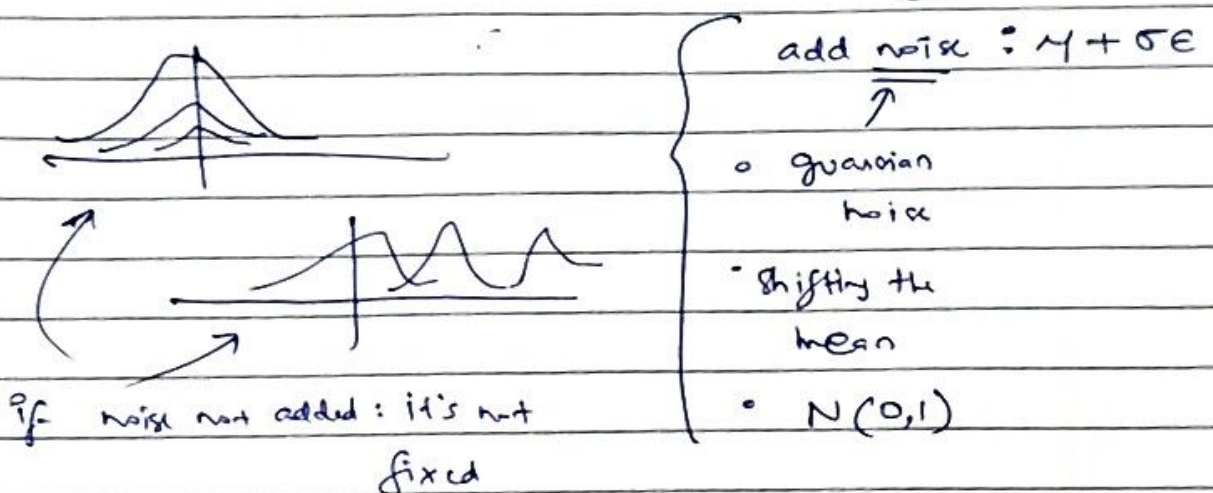
problem in variational autoencoder

 $\Downarrow$

① When doing backpropagation we can't determine which x was used

 $\Downarrow$

(that's why we use reparameterization)

 $\Downarrow$ 

but when noise added:

 $(0,1)$ (something fixed)

KL Divergence

(# loss fn) $\rightarrow$ for GAN ①

* Discriminator $\xrightarrow{\text{real data}} \max E_{x \sim p_r(x)} [\log D(x)]$

NOTE: $D(x)$ tells pb that sample is real

* Generator $\xrightarrow{\text{fake data}} \max E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$

$D(G(z))$: pb fake sample is real

* Generator

$\Rightarrow D(G(z)) = 0$ (on fake data $\rightarrow$ tell 0)

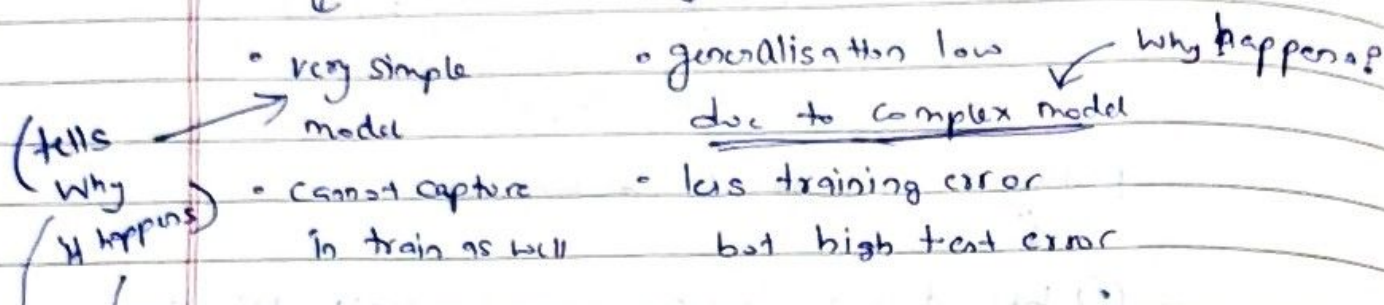
(we want it low)

(to make produce fake data as close to real)

$\min_G \max_D L(D, G) = ① + ②$

Day 28 (23/04/26)

underfitting and overfitting



how to handle?

→ increase complexity

→ underspecified data: i.e. actual data 100 features but re represented by 50

why overfitting? — complex model which noise ~~is~~ captured not related with the underlying data

Note:

$\hat{f}(x)$: my prediction
 $f(x)$: ~~the~~ true function

ground truth
 $y = f(x) + \epsilon$

error = bias + variance + irreducible

$(E[\hat{f}(x)] - f(x))^2$

$E[(\hat{f}(x) - E[\hat{f}(x)])^2]$

error $\rightarrow E[(y - \hat{f}(x))^2]$

• bias — error during training time

$E[\hat{f}(x)]$ means

- take the model
- train it on many different datasets
- average it's predictions

→ any model's error is bias

→ high in underfitting during training

→ less compared to overfitting in testing data

due to which low variance

as similar error on both dataset

→ overfitting —

low bias — (as training data) fit well
high variance

$$y - \hat{f}(x) = (f(x) - \hat{f}(x)) + e$$

$$(y - \hat{f}(x))^2 = (f(x) - \hat{f}(x))^2 + e^2 + 2e(f(x) - \hat{f}(x))$$

$$(y - \hat{f}(x))^2 = y^2 + \hat{f}(x)^2 - 2y \cdot \hat{f}(x)$$

$$E(y - \hat{f}(x))^2 = E[(f(x) - \hat{f}(x))^2] + E(e^2) = (f(x) - \hat{f}(x))^2 + \sigma^2$$

Bias - error betⁿ ground truth and predicted data

$$E[y - \hat{y}]$$

$$\text{Variance} = E[\hat{y} - E[\hat{y}]]$$

prediction from the current dataset

average prediction of different datasets

E_1

E_2, E_3

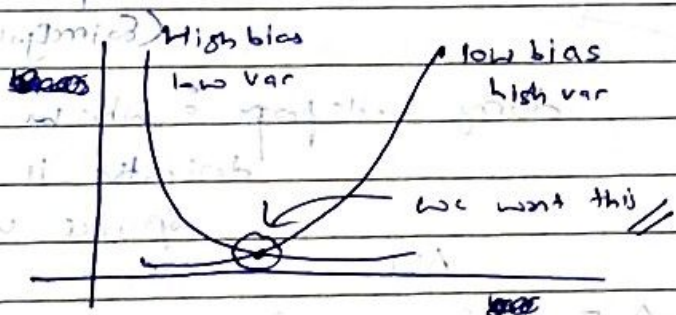
Total error

$$= \arg \min_{\theta} [\text{Bias}^2 + \text{Variance} + \text{Irr. error}]$$

$$= E \left[\left[y - \hat{f}(x) \right]^2 \right] \quad (\approx \text{MSE})$$

avg over all possible datasets / random ones

Trade off between bias and variance



How to handle

- increase Sample size
- feature selection
- cross validation
- k fold cross validation

What's n and y?

- early stop
- ↳ before overfitting occurs

(generally used)

- regularization
 - ↳ L1 (Lasso)
 - ↳ L2 (Ridge)

• dropout

• ensembling - Combining models !!

(is regularisation and
normalisation
(simp))

classmate

Date
Page

Regularization

↳ we try to control error

$$Cost = \min (\text{loss function}) + \text{penalty}$$

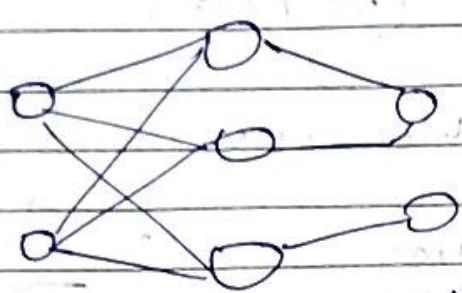
$$L2 \rightarrow \text{loss} = L + \lambda \sum_{i=1}^n \|w_i\|^2$$

λ : hyperparameter

$$\text{partial derivative} = \frac{\lambda \sum \|w_i\|^2}{n}$$

generalising

$$\text{loss} = L + \frac{\lambda}{2n} \sum_{l=1}^L \sum_i \sum_j \|w_{ij}\|^2$$



no. of layers

penalty

if $w_{ij} \uparrow \Rightarrow$

penalizing

(So, irregular wts)

during backprop: when we do partial derivative it is wrt specific w_{ij}

$$\Delta E = \frac{\partial L}{\partial w_{ij}} + \frac{\lambda}{n} w_{ij}$$

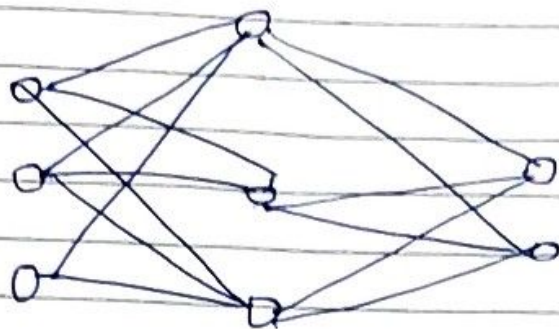
$$w_{ij}^t = w_{ij}^{t-1} - \eta \Delta w$$

$$= w_{ij}^{t-1} - \eta \left(\Delta w + \frac{\lambda}{n} w_{ij}^{t-1} \right)$$

$$= w_{ij}^{t-1} \left(1 - \frac{\lambda}{n} \right) - \eta \Delta w$$

In LI - we don't have squared term //

Dropout $\rightarrow$ (for overfitting)



- Randomly drops some of the nodes from each layer with some pb

• It's done only in training stage

• during test - overall network is considered

due to reduce there is a pb: Since 0.25 was dropped
So in a case: we have

classical dropout

to handle this:

0.75 of
params trained in
each epoch

training:

with pb $\leftarrow$

normal val with $1-E$

• we do $(1-p)$

In testing phase

• i.e. $op * (1-p)$ (guts) - (check)

$m = g(n)$
 $f(n) = \frac{m}{(1-E)}$
(Training) output

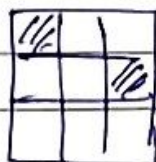
done by masking on each (w) $\leftarrow$ inverted dropout

that's why bring

testing: $(1-E) * \text{Full network output}$

(p)

i.e.



$W \leftarrow$ matrix

training: Scale by kept neurons

training output = $g(n) * m$

$(1-E)$

(NOTE: not present in class
cal dropout)

testing: do nothing //

$E = pb$ of dropping

$m = 0$ or 1

drop

keep